

LAPORAN PRATIKUM
STRUKTUR DATA
“Laporan Praktikum Pekan 5”

Disusun Oleh:

Zaki Misucha Andefi

2511533006

Dosen Pengampu : Dr. Wahyudi, S.T, M.T.

Asisten Praktikum : Sherly Sukmadira



DEPARTEMEN INFORMATIKA
FAKULTAS TEKNOLOGI INFORMASI
UNIVERSITAS ANDALAS

2026

KATA PENGANTAR

Puji dan syukur penulis panjatkan ke hadirat Tuhan Yang Maha Esa karena atas rahmat dan karunia-Nya laporan praktikum ini dapat diselesaikan dengan baik dan tepat waktu. Laporan ini disusun sebagai salah satu bentuk tugas dalam mata kuliah Struktur Data yang merupakan lanjutan dari pembelajaran Algoritma dan Pemrograman.

Pada praktikum ini, penulis mempelajari dan mengimplementasikan struktur data Single Linked List (SLL) menggunakan bahasa pemrograman Java. Berbagai contoh program dibuat untuk memahami konsep dasar, cara kerja, serta penerapan dari operasi-operasi pada SLL seperti penambahan node, pencarian data, dan penghapusan node.

Penulis menyadari bahwa dalam penyusunan laporan ini masih terdapat kekurangan, baik dari segi penulisan maupun isi materi. Oleh karena itu, penulis sangat mengharapkan kritik dan saran yang membangun demi perbaikan di masa yang akan datang.

Akhir kata, semoga laporan ini dapat memberikan manfaat serta menambah wawasan bagi pembaca dalam memahami konsep struktur data, khususnya Single Linked List.

Padang, April 2026

Penulis

DAFTAR ISI

KATA PENGANTAR	i
DAFTAR ISI.....	ii
BAB I PENDAHULUAN	1
1.1 Latar Belakang	1
1.2 Tujuan	1
1.3 Manfaat	1
BAB II PEMBAHASAN	2
2.1 Langkah Kerja.....	2
2.2 Program Iterasi Queue	2
2.3 Program Queue Array	3
2.4 Program Queue Array Driver.....	3
2.5 Program Queue Linked List.....	5
2.5 Program Reverse Data	5
BAB III KESIMPULAN.....	6
DAFTAR PUSTAKA	7

BAB I

PENDAHULUAN

1.1 Latar Belakang

Struktur data merupakan salah satu konsep penting dalam dunia pemrograman yang digunakan untuk mengorganisasi dan mengelola data agar dapat diakses secara efisien. Dalam pengembangan program, pemilihan struktur data yang tepat sangat berpengaruh terhadap kinerja dan efektivitas suatu sistem.

Single Linked List (SLL) adalah salah satu struktur data berbasis pointer yang terdiri dari kumpulan node, di mana setiap node menyimpan data dan pointer yang menunjuk ke node berikutnya. Node terakhir pada SLL memiliki pointer yang bernilai null, menandakan akhir dari list. Struktur ini sangat berguna untuk implementasi daftar dinamis yang ukurannya dapat berubah-ubah selama program berjalan.

Operasi dasar pada Single Linked List meliputi penambahan node (di depan, di belakang, maupun di posisi tertentu), pencarian data (traversal dan pencarian kunci), serta penghapusan node (head, tail, maupun di posisi tertentu). Pemahaman terhadap operasi-operasi ini menjadi fondasi penting dalam mempelajari struktur data yang lebih kompleks.

Dalam praktikum ini, dilakukan implementasi Single Linked List menggunakan bahasa pemrograman Java. Dengan demikian, diharapkan mahasiswa dapat memahami konsep dan penerapan SLL secara lebih mendalam.

1.2 Tujuan

Memahami konsep Single Linked List serta mengimplementasikan operasi penambahan, pencarian, dan penghapusan node dalam bahasa Java, serta melatih kemampuan dalam pengolahan data menggunakan struktur data berbasis pointer.

1.3 Manfaat

Meningkatkan pemahaman tentang struktur data Single Linked List, melatih logika pemrograman berbasis pointer, serta memberikan pengalaman dalam implementasi operasi-operasi dasar pada linked list seperti penambahan, pencarian, dan penghapusan node.

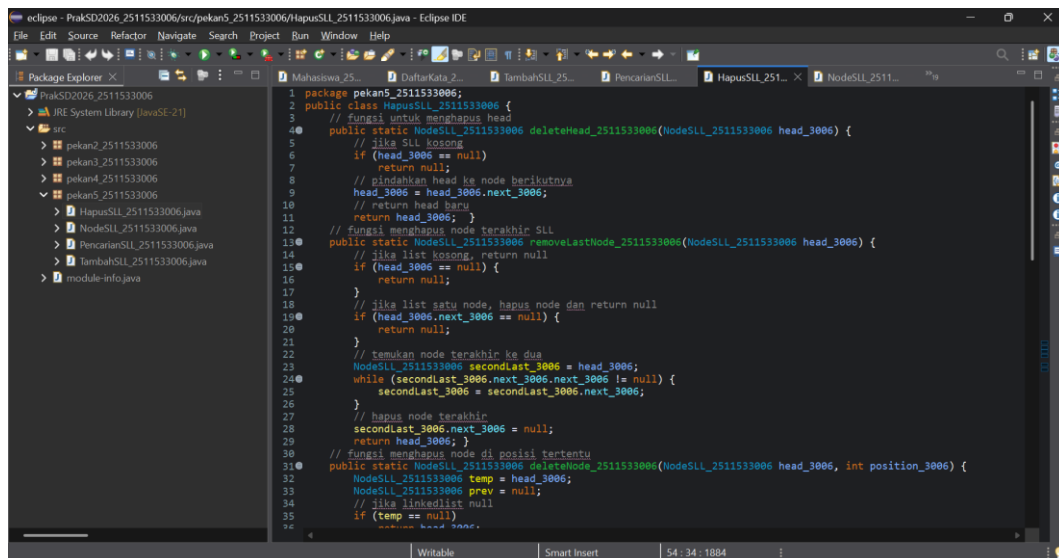
BAB II

PEMBAHASAN

2.1 Langkah Kerja

1. Membuka IDE (Eclipse).
2. Membuat package baru bernama pekan4_2511533006.
3. Menambahkan beberapa class:
 - NodeSLL_2511533006
 - TambahSLL_2511533006
 - PencarianSLL_2511533006
 - HapusSLL_2511533006
4. Menjalankan program satu per satu dan mengamati hasilnya.

2.2 Program Hapus SLL



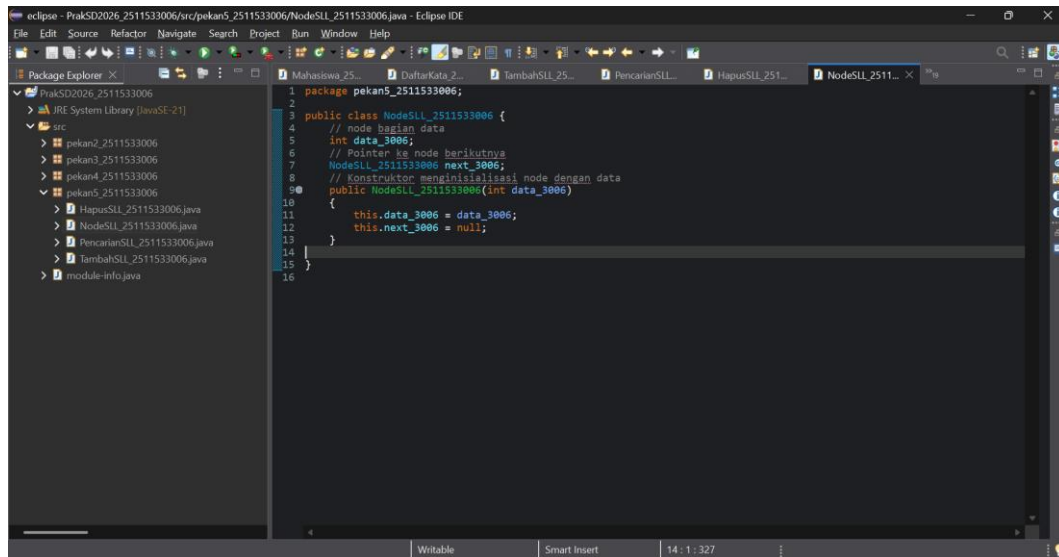
```
1 package pekan5_2511533006;
2 public class HapusSLL_2511533006 {
3     // fungsi untuk menghapus head
4     public static NodeSLL_2511533006 deleteHead_2511533006(NodeSLL_2511533006 head_3006) {
5         // jika SLL kosong
6         if (head_3006 == null)
7             return null;
8         // pindahkan head ke node berikutnya
9         head_3006 = head_3006.next_3006;
10        // return head baru
11        return head_3006;
12    }
13    // fungsi menghapus node terakhir SLL
14    public static NodeSLL_2511533006 removeLastNode_2511533006(NodeSLL_2511533006 head_3006) {
15        // jika list kosong, return null
16        if (head_3006 == null) {
17            return null;
18        }
19        // jika list satu node, hapus node dan return null
20        if (head_3006.next_3006 == null) {
21            return null;
22        }
23        // temukan node terakhir ke dua
24        NodeSLL_2511533006 secondLast_3006 = head_3006;
25        while (secondLast_3006.next_3006.next_3006 != null) {
26            secondLast_3006 = secondLast_3006.next_3006;
27        }
28        // hapus node terakhir
29        secondLast_3006.next_3006 = null;
30        return head_3006;
31    }
32    // fungsi menghapus node di posisi tertentu
33    public static NodeSLL_2511533006 deleteNode_2511533006(NodeSLL_2511533006 head_3006, int position_3006) {
34        NodeSLL_2511533006 temp = head_3006;
35        NodeSLL_2511533006 prev = null;
36        // jika linkedlist null
37        if (temp == null)
38            return head_3006;
39    }
40 }
```

Class HapusSLL_2511533006 mengimplementasikan tiga metode penghapusan node pada Single Linked List, yaitu penghapusan node head (`deleteHead_2511533006`), penghapusan node terakhir (`removeLastNode_2511533006`), dan penghapusan node di posisi tertentu (`deleteNode_2511533006`).

Analisis: Program ini mendemonstrasikan tiga cara penghapusan node pada SLL. Metode `deleteHead_2511533006` memindahkan pointer head ke node berikutnya sehingga node pertama terhapus. Metode `removeLastNode_2511533006` menelusuri list hingga menemukan node kedua dari terakhir (`secondLast`), kemudian mengatur pointer next-nya menjadi null untuk menghapus node terakhir. Metode `deleteNode_2511533006` menghapus node di posisi tertentu dengan cara menelusuri list ke posisi tersebut, lalu menghubungkan node sebelumnya langsung ke node setelahnya. Pada program ini, list awal 1-->2-->3-->4--

->5-->6, setelah menghapus node terakhir menjadi 1-->2-->3-->4-->5, dan setelah menghapus posisi ke-2 menjadi 1-->3-->4-->5.

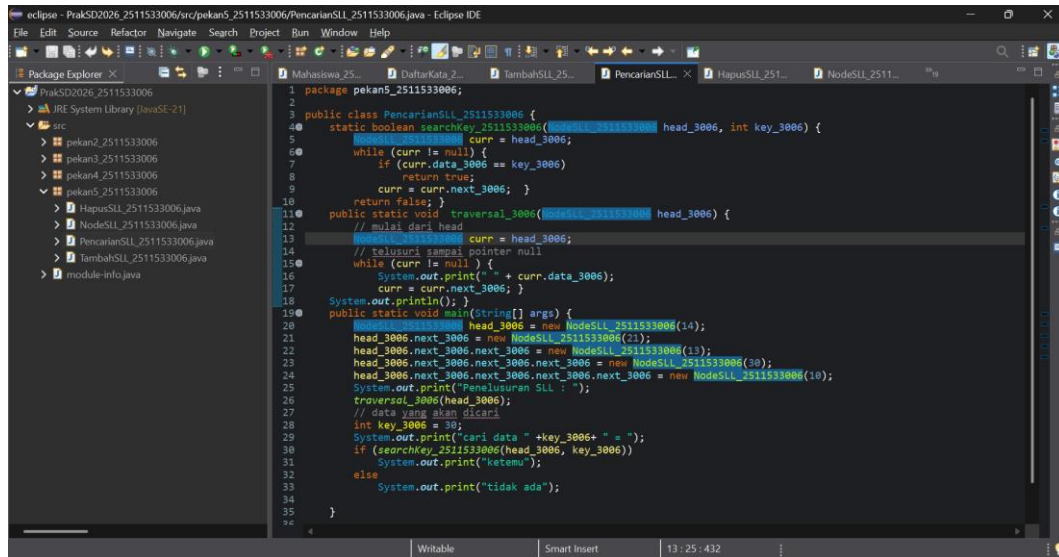
2.3 Program Node SLL



Class NodeSLL_2511533006 merupakan struktur dasar dari Single Linked List. Setiap node terdiri dari dua bagian: bagian data yang menyimpan nilai integer, dan pointer next_3006 yang menunjuk ke node berikutnya. Konstruktor digunakan untuk menginisialisasi node dengan nilai data yang diberikan dan mengatur pointer next menjadi null.

Analisis: Class ini berfungsi sebagai blueprint (cetak biru) dari setiap simpul (node) dalam Single Linked List. Atribut data_3006 menyimpan nilai integer dari node, sedangkan next_3006 adalah referensi ke node berikutnya dalam list. Ketika konstruktor dipanggil, pointer next secara otomatis diinisialisasi ke null yang berarti node belum terhubung ke node manapun.

2.4 Program Pencarian SLL

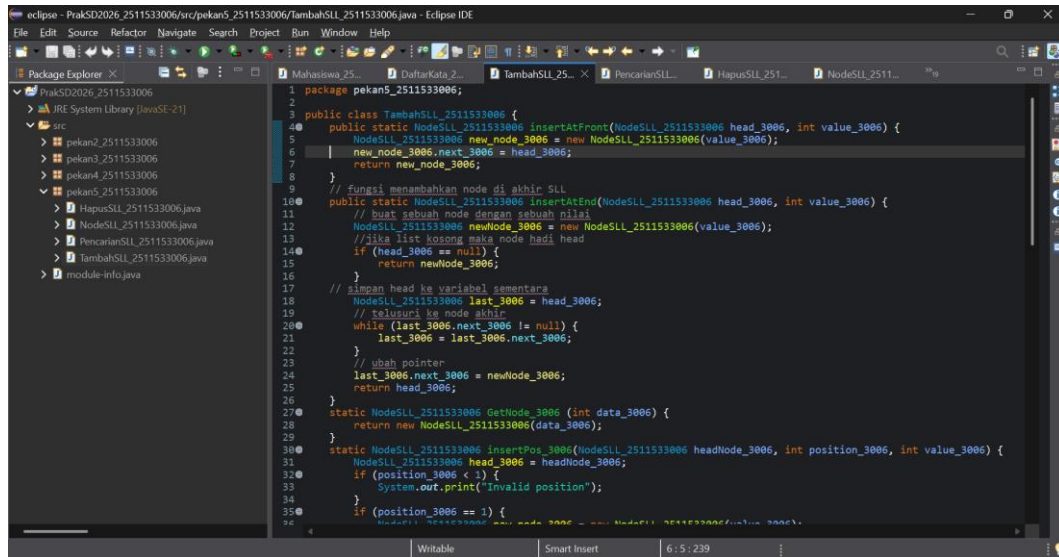


```
1 package pekan5_2511533006;
2
3 public class PencarianSLL_2511533006 {
4     static boolean searchKey_2511533006(NodeSLL_2511533006 head_3006, int key_3006) {
5         NodeSLL_2511533006 curr = head_3006;
6         while (curr != null) {
7             if (curr.data_3006 == key_3006)
8                 return true;
9             curr = curr.next_3006;
10        }
11        return false;
12    }
13
14    public static void traversal_3006(NodeSLL_2511533006 head_3006) {
15        // mulai dari head
16        NodeSLL_2511533006 curr = head_3006;
17        // telusuri sampai pointer null
18        while (curr != null) {
19            System.out.print(" " + curr.data_3006);
20            curr = curr.next_3006;
21        }
22        System.out.println();
23    }
24
25    public static void main(String[] args) {
26        NodeSLL_2511533006 head_3006 = new NodeSLL_2511533006(14);
27        head_3006.next_3006 = new NodeSLL_2511533006(21);
28        head_3006.next_3006.next_3006 = new NodeSLL_2511533006(13);
29        head_3006.next_3006.next_3006.next_3006 = new NodeSLL_2511533006(30);
30        head_3006.next_3006.next_3006.next_3006.next_3006 = new NodeSLL_2511533006(10);
31        System.out.print("Penelusuran SLL : ");
32        traversal_3006(head_3006);
33        // data yang akan dicari
34        int key_3006 = 30;
35        System.out.print("cari data " + key_3006 + " = ");
36        if (searchKey_2511533006(head_3006, key_3006))
37            System.out.print("ketemu");
38        else
39            System.out.print("tidak ada");
40    }
41 }
```

Class `PencarianSLL_2511533006` mengimplementasikan dua operasi pencarian pada Single Linked List, yaitu traversal untuk menelusuri seluruh node dan `searchKey` untuk mencari data tertentu dalam list.

Analisis: Program ini mendemonstrasikan dua operasi penting pada SLL. Metode `traversal_3006` menelusuri seluruh node dari head hingga null dan mencetak nilai setiap node. Metode `searchKey_2511533006` melakukan pencarian linear (linear search) dengan membandingkan setiap nilai node dengan kunci yang dicari. Jika ditemukan, metode mengembalikan `true`; jika tidak ditemukan hingga akhir list, mengembalikan `false`. Pada program ini, SLL berisi nilai 14, 21, 13, 30, 10, dan program mencari nilai 30 yang hasilnya adalah "ketemu".

2.5 Program Tambah SLL



```
1 package pekan5_2511533006;
2
3 public class TambahSLL_2511533006 {
4     public static NodeSLL_2511533006 insertAtFront(NodeSLL_2511533006 head_3006, int value_3006) {
5         NodeSLL_2511533006 new_node_3006 = new NodeSLL_2511533006(value_3006);
6         new_node_3006.next_3006 = head_3006;
7         return new_node_3006;
8     }
9     // fungsi menambahkan node di akhir SLL
10    public static NodeSLL_2511533006 insertAtEnd(NodeSLL_2511533006 head_3006, int value_3006) {
11        // buat sebuah node dengan sebuah nilai
12        NodeSLL_2511533006 newNode_3006 = new NodeSLL_2511533006(value_3006);
13        // jika list kosong maka node jadi head
14        if (head_3006 == null) {
15            return newNode_3006;
16        }
17        // simpan head ke variabel sementara
18        NodeSLL_2511533006 last_3006 = head_3006;
19        // telusuri ke node akhir
20        while (last_3006.next_3006 != null) {
21            last_3006 = last_3006.next_3006;
22        }
23        // ubah pointer
24        last_3006.next_3006 = newNode_3006;
25        return head_3006;
26    }
27    static NodeSLL_2511533006 getNode_3006(int data_3006) {
28        return new NodeSLL_2511533006(data_3006);
29    }
30    static NodeSLL_2511533006 insertPos_3006(NodeSLL_2511533006 headNode_3006, int position_3006, int value_3006) {
31        NodeSLL_2511533006 head_3006 = headNode_3006;
32        if (position_3006 < 1) {
33            System.out.print("Invalid position");
34        }
35        if (position_3006 == 1) {
36            return insertAtFront(head_3006, value_3006);
37        }
38    }
39 }
```

Class TambahSLL_2511533006 mengimplementasikan tiga metode penambahan node pada Single Linked List, yaitu penambahan di depan (insertAtFront), di belakang (insertAtEnd), dan di posisi tertentu (insertPos_3006).

Analisis: Program ini mendemonstrasikan tiga cara penambahan node pada SLL. Metode insertAtFront membuat node baru dan menghubungkannya sebelum head, sehingga node baru menjadi head yang baru. Metode insertAtEnd menelusuri list hingga node terakhir, kemudian menambahkan node baru di akhir. Metode insertPos_3006 menambahkan node di posisi tertentu dengan menelusuri list hingga posisi yang dituju. Hasil output program menunjukkan list 2-->3-->5-->6, kemudian menjadi 1-->2-->3-->5-->6 setelah menambah di depan, lalu 1-->2-->3-->5-->6-->7 setelah menambah di belakang, dan terakhir 1-->2-->3-->4-->5-->6-->7 setelah menambah angka 4 di posisi ke-4.

BAB III

KESIMPULAN

Single Linked List (SLL) merupakan struktur data yang sangat fundamental dalam pemrograman karena menyediakan cara yang fleksibel dan efisien untuk menyimpan dan mengelola kumpulan data secara dinamis. Tidak seperti array yang memiliki ukuran tetap, SLL dapat berkembang dan menyusut secara dinamis sesuai kebutuhan program.

Melalui praktikum ini, dapat dipahami bahwa SLL terdiri dari node-node yang saling terhubung melalui pointer. Operasi penambahan node pada SLL dapat dilakukan di tiga posisi yaitu di depan (head), di belakang (tail), maupun di posisi tertentu. Setiap metode penambahan memiliki kompleksitas yang berbeda, di mana penambahan di depan memiliki kompleksitas $O(1)$ sementara penambahan di belakang dan di posisi tertentu memiliki kompleksitas $O(n)$ karena perlu menelusuri list terlebih dahulu.

Operasi pencarian pada SLL menggunakan algoritma pencarian linear yang menelusuri node dari head hingga tail. Jika data yang dicari ditemukan, maka pencarian berhenti dan mengembalikan nilai true. Kompleksitas pencarian ini adalah $O(n)$ dalam kasus terburuk.

Operasi penghapusan node pada SLL juga dapat dilakukan pada tiga posisi yaitu node head, node tail, dan node di posisi tertentu. Penghapusan head memiliki kompleksitas $O(1)$, sementara penghapusan tail dan posisi tertentu memiliki kompleksitas $O(n)$. Hal yang perlu diperhatikan dalam penghapusan adalah memastikan pointer dari node sebelumnya diperbarui dengan benar agar tidak terjadi memory leak atau list yang terputus.

Dengan memahami konsep dan implementasi Single Linked List beserta operasi-operasinya, mahasiswa diharapkan mampu mengaplikasikannya dalam pembuatan program yang lebih kompleks serta meningkatkan kemampuan dalam mengelola dan memproses data secara efektif dan efisien.

DAFTAR PUSTAKA

- [1] H. M. Deitel dan P. J. Deitel, *Java: How to Program*, 11th ed. New Jersey, NJ: Pearson, 2017.

Tugas Struktur Data

1. Pasien_2511533006

```
package pekan5_2511533006;

//=====
//Kelas ADT Pasien - Single Linked List Node
//Tugas Praktikum Pekan 5: Struktur Data
//NIM: 2511533006
//=====

public class Pasien_2511533006 {

    // — Atribut —————
    private String namaPasien_3006;
    private String penyakit_3006;
    private int nomorAntrian_3006;

    // Pointer ke node Pasien berikutnya
    public Pasien_2511533006 next_3006;

    // — Constructor —————
    /**
     * Menginisialisasi semua atribut node Pasien.
     *
     * @param nama      Nama pasien
     * @param keluhan   Keluhan / penyakit pasien
     * @param nomor     Nomor antrian pasien
     */
    public Pasien_2511533006(String nama, String keluhan, int nomor) {
        this.namaPasien_3006 = nama;
        this.penyakit_3006 = keluhan;
        this.nomorAntrian_3006 = nomor;
        this.next_3006 = null; // pointer awal selalu null
    }

    // — Getter (Selektor) —————
    public String getNamaPasien_3006() {
        return namaPasien_3006;
    }

    public String getPenyakit_3006() {
        return penyakit_3006;
    }

    public int getNomorAntrian_3006() {
        return nomorAntrian_3006;
    }

    public Pasien_2511533006 getNext_3006() {
        return next_3006;
    }

    // — Setter (Mutator) —————
    public void setNamaPasien_3006(String nama) {
```

```

        this.namaPasien_3006 = nama;
    }

    public void setPenyakit_3006(String keluhan) {
        this.penyakit_3006 = keluhan;
    }

    public void setNomorAntrian_3006(int nomor) {
        this.nomorAntrian_3006 = nomor;
    }

    public void setNext_3006(Pasien_2511533006 next) {
        this.next_3006 = next;
    }
}

```

2. RumahSakit_2511533006

```

package pekan5_2511533006;

//=====
//Kelas Driver Rumah Sakit - Simulasi Antrian (Single Linked List)
//Tugas Praktikum Pekan 5: Struktur Data
//NIM: 2511533006
//=====

import java.util.Scanner;

public class RumahSakit_2511533006 {

    // — Field utama linked list —
    private static Pasien_2511533006 head_3006 = null; // kepala
    antrian
    private static int counterAntrian_3006 = 0; // auto-
    increment nomor antrian

    // =====
    // 1. DAFTARKAN PASIEN – Insert at Tail
    // Menambahkan node baru di akhir linked list (FIFO).
    // =====
    private static void daftarkanPasien_3006(Scanner sc) {
        System.out.print("Masukkan Nama Pasien : ");
        String nama_3006 = sc.nextLine();

        System.out.print("Masukkan Keluhan : ");
        String keluhan_3006 = sc.nextLine();

        // Auto-increment nomor antrian
        counterAntrian_3006++;

        // Buat node baru
        Pasien_2511533006 nodeBaru_3006 = new Pasien_2511533006(
            nama_3006, keluhan_3006, counterAntrian_3006);

        // Jika list kosong → node langsung jadi head
        if (head_3006 == null) {

```

```

        head_3006 = nodeBaru_3006;
    } else {
        // Telusuri sampai node terakhir (next == null)
        Pasien_2511533006 temp_3006 = head_3006;
        while (temp_3006.next_3006 != null) {
            temp_3006 = temp_3006.next_3006;
        }
        // Sambungkan node baru di ekor
        temp_3006.next_3006 = nodeBaru_3006;
    }

    System.out.println(" Pasien berhasil didaftarkan! Nomor
Antrian: "
        + counterAntrian_3006);
}

// =====
// 2. PANGGIL PASIEN – Delete Head
// Menghapus node terdepan dan menampilkan datanya.
// =====
private static void panggilPasien_3006() {
    // Cek kondisi list kosong
    if (head_3006 == null) {
        System.out.println(" Antrian kosong. Tidak ada pasien
untuk dipanggil.");
        return;
    }

    // Simpan data pasien yang dipanggil sebelum dihapus
    Pasien_2511533006 dipanggil_3006 = head_3006;

    // Geser head ke node berikutnya
    head_3006 = head_3006.next_3006;

    // Putuskan pointer agar node lama bisa di-GC
    dipanggil_3006.next_3006 = null;

    System.out.println("=====");
    System.out.println(" PASIEN DIPANGGIL");
    System.out.println(" No. Antrian : " +
dipanggil_3006.getNomorAntrian_3006());
    System.out.println(" Nama : " +
dipanggil_3006.getNamaPasien_3006());
    System.out.println(" Keluhan : " +
dipanggil_3006.getPenyakit_3006());
    System.out.println("=====");
}

// =====
// 3. TAMPILKAN ANTRIAN – Display / Traverse
// Menelusuri linked list dari head hingga null.
// =====
private static void tampilkanAntrian_3006() {
    // Cek kondisi list kosong
    if (head_3006 == null) {
        System.out.println("⚠ Antrian kosong. Tidak ada pasien
dalam antrian.");
    }
}

```

```

        return;
    }

    System.out.println("=====
=====");
    System.out.println("                DAFTAR ANTRIAN PASIEN");

    System.out.println("=====
=====");
    System.out.printf(" %-5s %-20s %-20s\n", "No.", "Nama Pasien",
"Keluhan");
    System.out.println("=====");

    // Telusuri list dari head sampai null
    Pasien_2511533006 current_3006 = head_3006;
    int posisi_3006 = 1;

    while (current_3006 != null) {
        System.out.printf(" %-5d %-20s %-20s\n",
            posisi_3006,
            current_3006.getNamaPasien_3006(),
            current_3006.getPenyakit_3006());
        current_3006 = current_3006.next_3006; // pindah ke node
berikutnya
        posisi_3006++;
    }

    System.out.println("=====
=====");
}

// =====
// 4. CARI PASIEN – Search (Case-Insensitive)
// Pencarian berdasarkan nama pasien.
// =====
private static void cariPasien_3006(Scanner sc) {
    System.out.print("Masukkan Nama Pasien yang dicari : ");
    String keyword_3006 = sc.nextLine().trim();

    boolean ditemukan_3006 = false;

    // Telusuri list dari head sampai null
    Pasien_2511533006 current_3006 = head_3006;
    int posisi_3006 = 1;

    while (current_3006 != null) {
        // Bandingkan nama secara case-insensitive
        if (current_3006.getNamaPasien_3006()
            .equalsIgnoreCase(keyword_3006)) {

            if (!ditemukan_3006) {

System.out.println("=====
=====");

```

```

        System.out.println("    HASIL PENCARIAN: \"\" +
keyword_3006 + "\"");

System.out.println("=====
=====");
    }

    System.out.println("    Posisi dalam antrian : \" +
posisi_3006);
    System.out.println("    No. Antrian          : \" +
current_3006.getNomorAntrian_3006());
    System.out.println("    Nama Pasien          : \" +
current_3006.getNamaPasien_3006());
    System.out.println("    Keluhan          : \" +
current_3006.getPenyakit_3006());
    System.out.println("=====");

    ditemukan_3006 = true;
}

    current_3006 = current_3006.next_3006; // pindah ke node
berikutnya
    posisi_3006++;
}

    if (!ditemukan_3006) {
        System.out.println("    Pasien dengan nama \"\" + keyword_3006
+ "\" tidak ditemukan.");
    } else {

System.out.println("=====
=====");
    }
}

// =====
// 5. CEK STATUS ANTRIAN
// Menampilkan jumlah total pasien & informasi pasien terdepan.
// =====
private static void cekStatusAntrian_3006() {
    // Cek kondisi list kosong
    if (head_3006 == null) {
        System.out.println("    Antrian kosong. Tidak ada pasien saat
ini.");
        return;
    }

    // Hitung jumlah node
    int total_3006 = 0;
    Pasien_2511533006 current_3006 = head_3006;
    while (current_3006 != null) {
        total_3006++;
        current_3006 = current_3006.next_3006;
    }

    System.out.println("=====
=====");
}

```

```

        System.out.println("STATUS ANTRIAN");
        System.out.println("  Total Pasien    : " + total_3006);
        System.out.println("  _____");
        System.out.println("  Pasien Terdepan :");
        System.out.println("  No. Antrian    : " +
head_3006.getNomorAntrian_3006());
        System.out.println("  Nama           : " +
head_3006.getNamaPasien_3006());
        System.out.println("  Keluhan        : " +
head_3006.getPenyakit_3006());
        System.out.println("  _____");
    }

    // =====
    // MAIN - Menu utama program
    // =====
    public static void main(String[] args) {
        Scanner scanner_3006 = new Scanner(System.in);
        int pilihan_3006;

        System.out.println("=====
        ");
        System.out.println("      Antrian Rumah Sakit - NIM:
        2511533006");

        System.out.println("=====
        ");

        do {
            // Tampilkan menu

            System.out.println("\n=====");
            System.out.println("      MENU      ");
            System.out.println("=====");
            System.out.println("  1. Daftarkan Pasien  (Insert)  ");
            System.out.println("  2. Panggil Pasien    (Delete)   ");
            System.out.println("  3. Tampilkan Antrian (Display)   ");
            System.out.println("  4. Cari Pasien       (Search)    ");
            System.out.println("  5. Cek Status Antrian ");
            System.out.println("  6. Keluar            ");
            System.out.println("=====");
            System.out.print("Pilihan: ");

            // Validasi input menu
            while (!scanner_3006.hasNextInt()) {
                System.out.print("Input tidak valid. Masukkan angka 1-
6: ");
                scanner_3006.next();
            }
            pilihan_3006 = scanner_3006.nextInt();
            scanner_3006.nextLine(); // buang newline sisa

            System.out.println();

            // Routing menu
            switch (pilihan_3006) {

```

```

        case 1: daftarkanPasien_3006(scanner_3006); break;
        case 2: panggilPasien_3006(); break;
        case 3: tampilkanAntrian_3006(); break;
        case 4: cariPasien_3006(scanner_3006); break;
        case 5: cekStatusAntrian_3006(); break;
        case 6:
            System.out.println(" Terima kasih. Program
selesai.");
            break;
        default:
            System.out.println(" Pilihan tidak valid. Masukkan
angka 1-6.");
    }

    } while (pilihan_3006 != 6);

    scanner_3006.close();
}
}

```

3. Output

```

RumahSakit_2511533006 [Java Application] C:\Users\Kii's\Downloads\eclipse-jav
Antrian Rumah Sakit - NIM: 2511533006

MENU
1. Daftarkan Pasien (Insert)
2. Panggil Pasien (Delete)
3. Tampilkan Antrian (Display)
4. Cari Pasien (Search)
5. Cek Status Antrian
6. Keluar

Pilihan: 1

Masukkan Nama Pasien : yanti
Masukkan Keluhan : maag
Pasien berhasil didaftarkan! Nomor Antrian: 1

MENU
1. Daftarkan Pasien (Insert)
2. Panggil Pasien (Delete)
3. Tampilkan Antrian (Display)
4. Cari Pasien (Search)
5. Cek Status Antrian
6. Keluar

Pilihan:

```